Stage 2. The ENIGMA Simulator

PhD. Elias Del Pozo

Departamento de Arquitectura y Tecnología de Sistemas InformáticosUniversidad Politecnica de Madrid, Spain
e.delpozo@upm.es

Prof. Felix Garcia Carballeira

Computer Science and Engineering Department
University Carlos III of Madrid, Spain
fgcarbal@inf.uc3m.es



Simulation Methodology for Edge, Fog, and Cloud Environments

- Scalability: A real IoT system may include thousands of sensors and devices, making manual evaluation or evaluation with inefficient tools unfeasible.
- Dynamic behavior: Devices change their sending frequency, data generation, and resource consumption depending on context or time.
- Mobility: Many IoT elements (vehicles, people, drones, ...) move over time, affecting connectivity, latency, and network load.
- Necessity: There is a need for a tool that can simulate these complex environemnts, anticipating problems and optimizing deployments before implementing them in the real world.



ENIGMA - gENeric Iot edGe siMulAtor

- A C++17 toolkit to model Edge, Fog and Cloud infrastructures and the applications that run on them.
- Available at: https://github.com/ENIGMA-Sim/ENIGMA

- Built on SimGrid 4.1 — a validated, scalable discrete-event simulator for distributed systems.

- Generates the SimGrid platform XML for you, ships ready-made example apps, adds two optional modules:
  1. enigma_mqtt — MQTT publish/subscribe (broker, publisher, subscriber)
  2. enigma_mobility — GPS traces attached to hosts, position interpolation, interactive maps

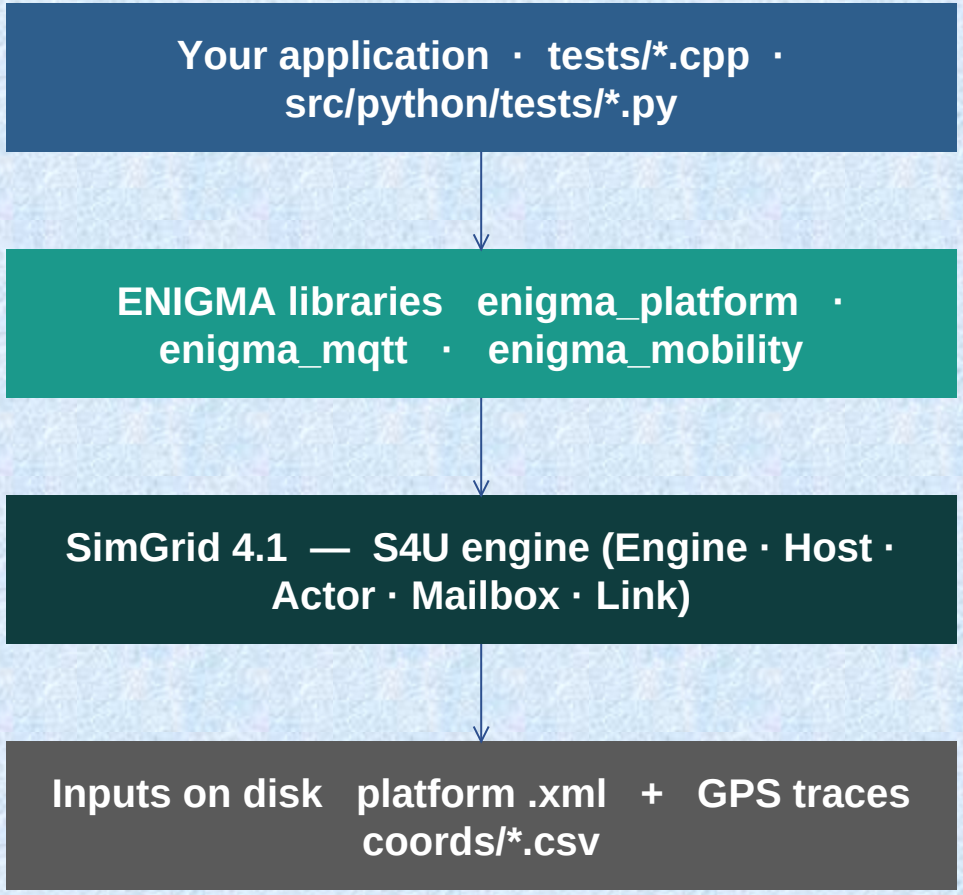
- One C++ API and a mirrored Python API — same platform XML, same traces, swap only the language.

DockerHub Repository

GitHub Repository

ENIGMA - Architecture

- Simulates Edge, Fog, and Cloud infrastructures to analyze CPU usage and power consumption.
- Helps to choose optimal environments according to user needs.
- Based on SimGrid, with support for data transmission between components.
- A mobility API was added to SimGrid for Iot movement.



DockerHub Repository

GitHub Repository

ENIGMA - Architecture

- Simulates Edge, Fog, and Cloud infrastructures to analyze CPU usage and power consumption.
- Helps to choose optimal environments according to user needs.
- Based on SimGrid, with support for data transmission between components.
- A mobility API was added to SimGrid for Iot movement.

platform_generator / PlatformBuilder → platforms/xxx.xml

↓

Engine e; e.load_platform(xml) — hosts, links, routing, mobility_dir

↓

deploy actors: host->add_actor(name, Callable) (one process per host)

↓

e.run() — discrete-event loop advances simulated time

↓

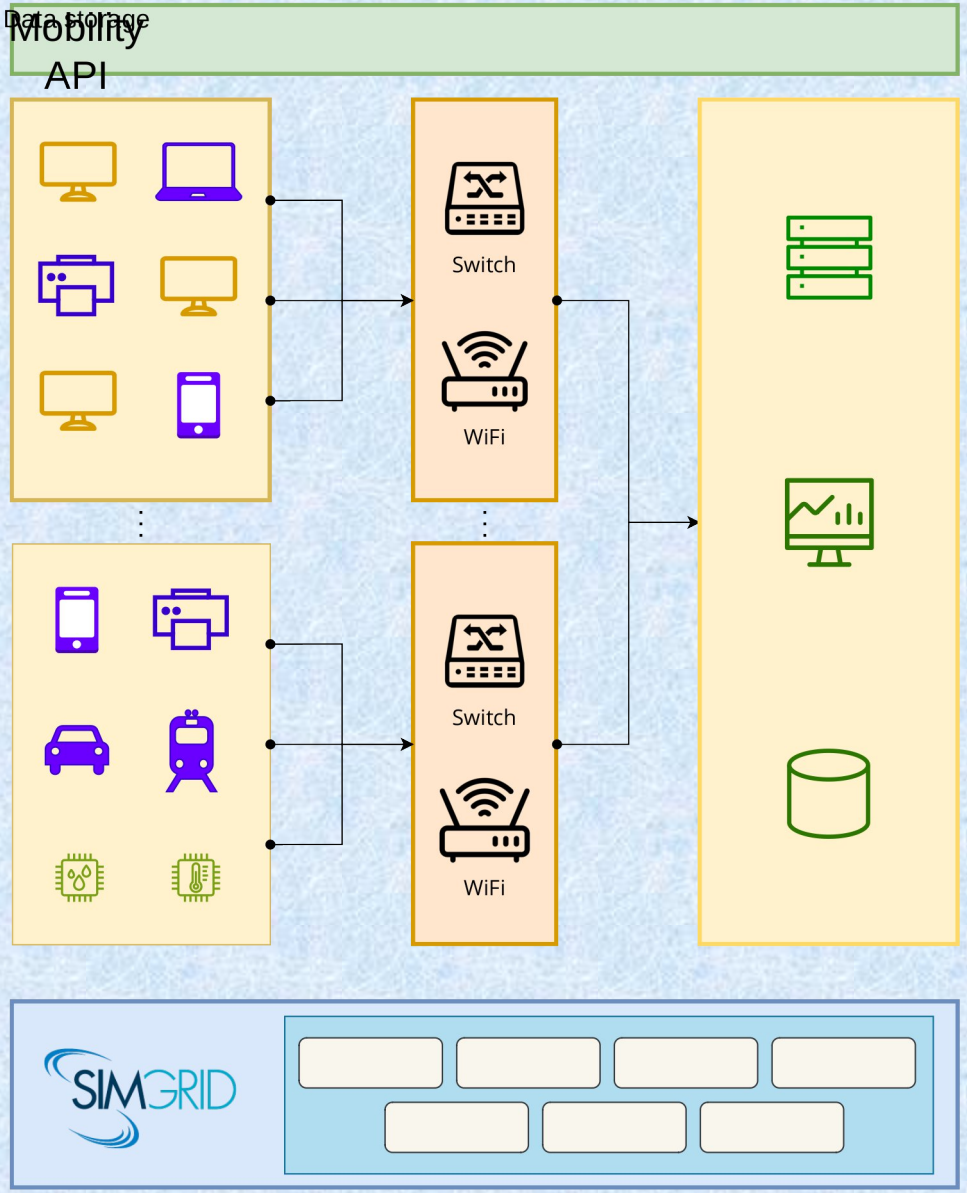
exports: logs (XBT_INFO) · mobility snapshots.json / .csv · interactive map

DockerHub Repository

GitHub Repository

ENIGMA - Architecture

- Simulates Edge, Fog, and Cloud infrastructures to analyze CPU usage and power consumption.
- Helps to choose optimal environments according to user needs.
- Based on SimGrid, with support for data transmission between components.
- A mobility API was added to SimGrid for Iot movement.

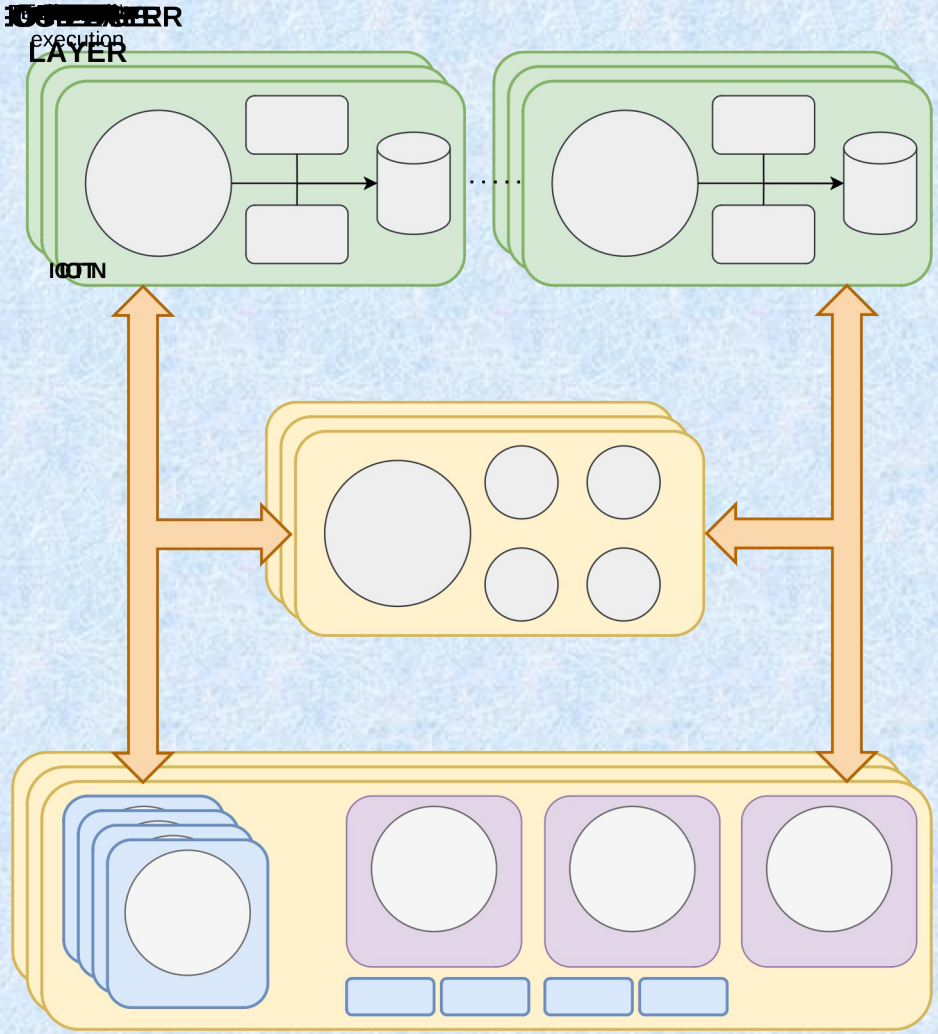


DockerHub Repository

GitHub Repository

ENIGMA - Deployment Design

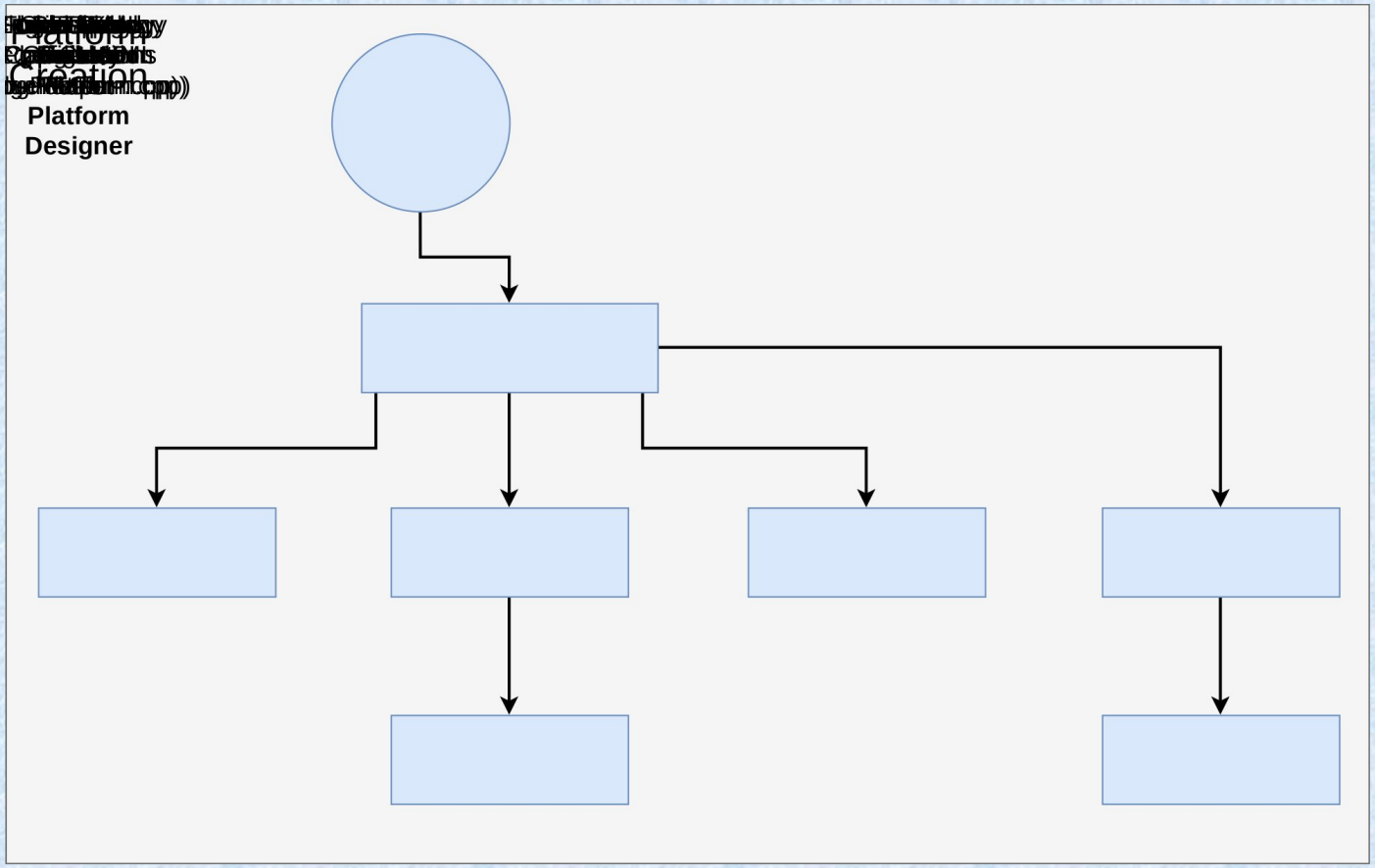
- Cloud Layer - To process tasks sent by IoT devices.
- Fog Layer - To reduce latency and offload the cloud.
- IoT-Edge Layer - To improve response times due to their proximity to the client.

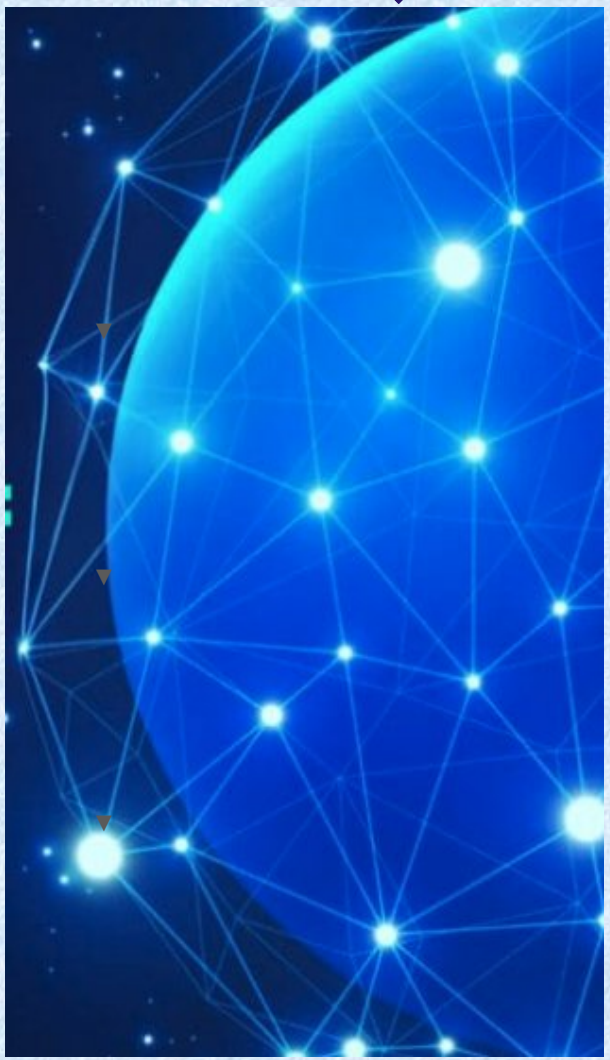


DockerHub Repository

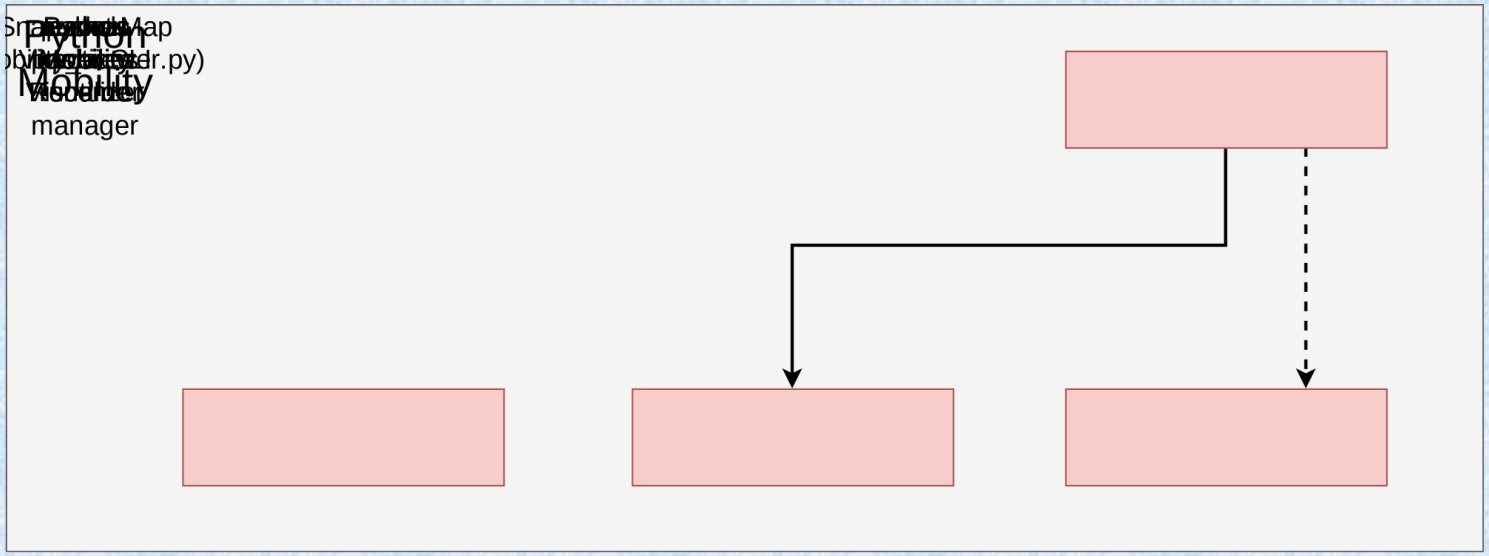
GitHub Repository

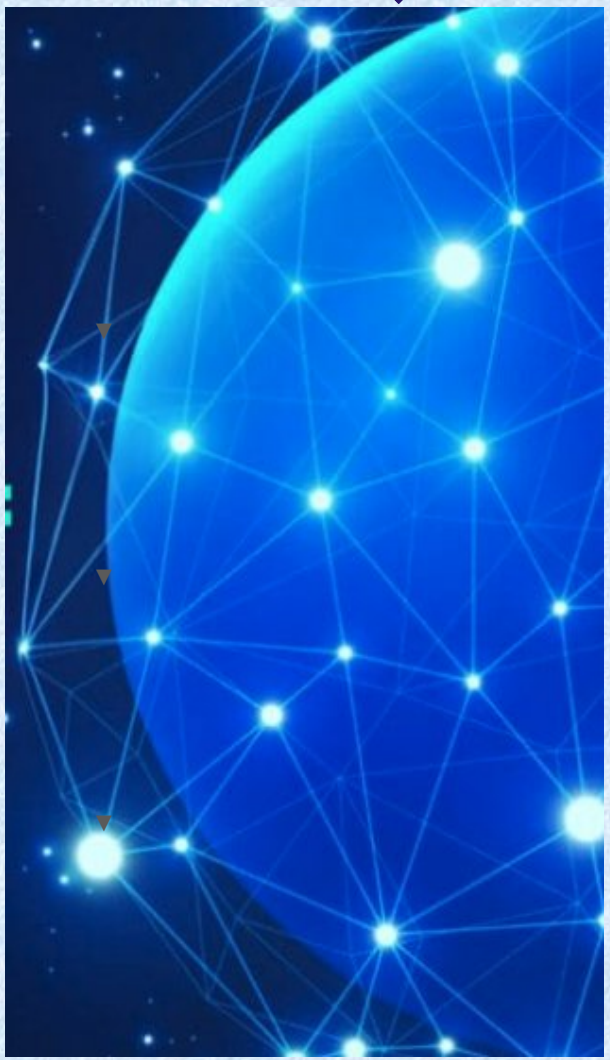
ENIGMA – GitHub Design – Platform Creation



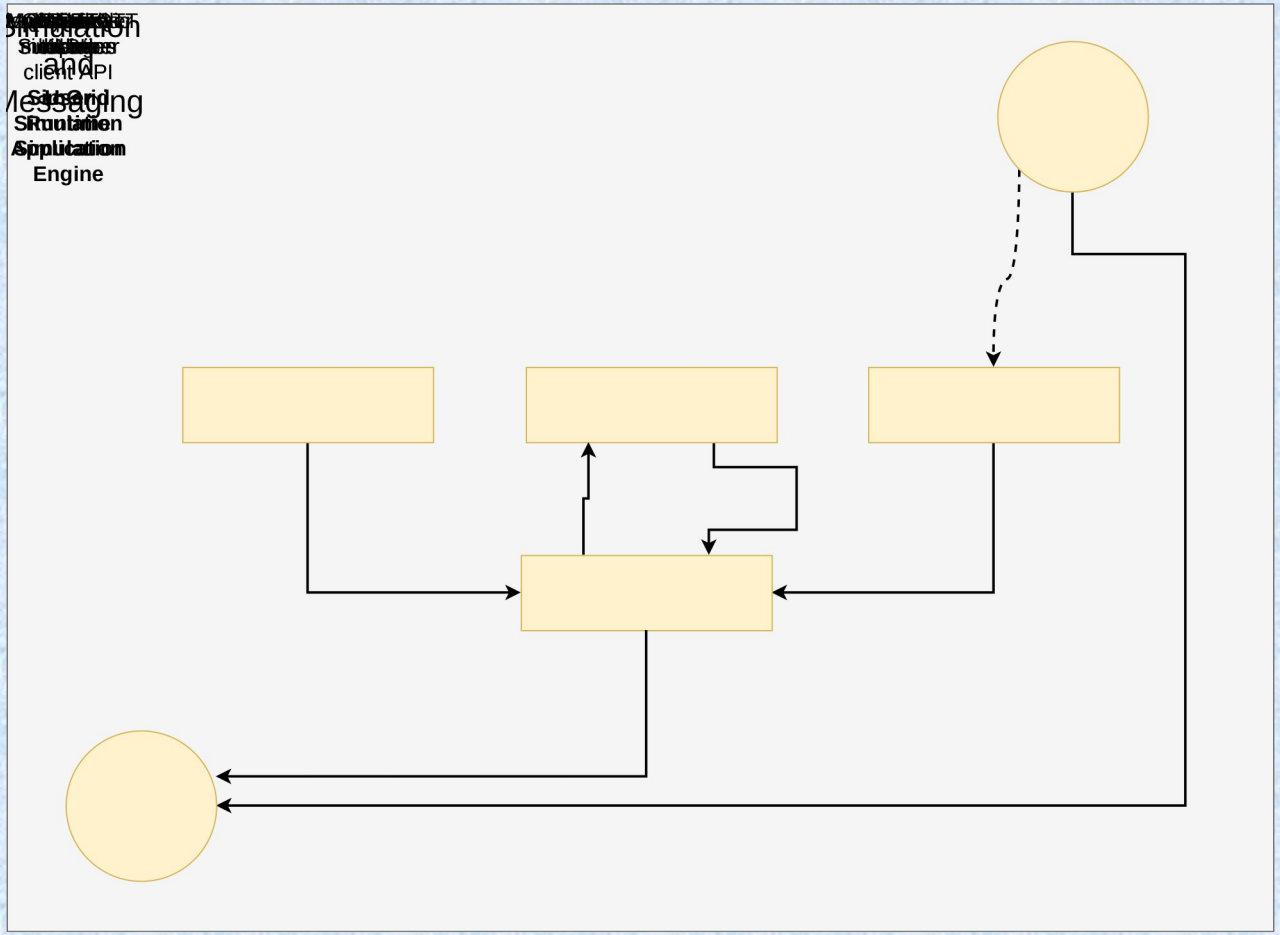


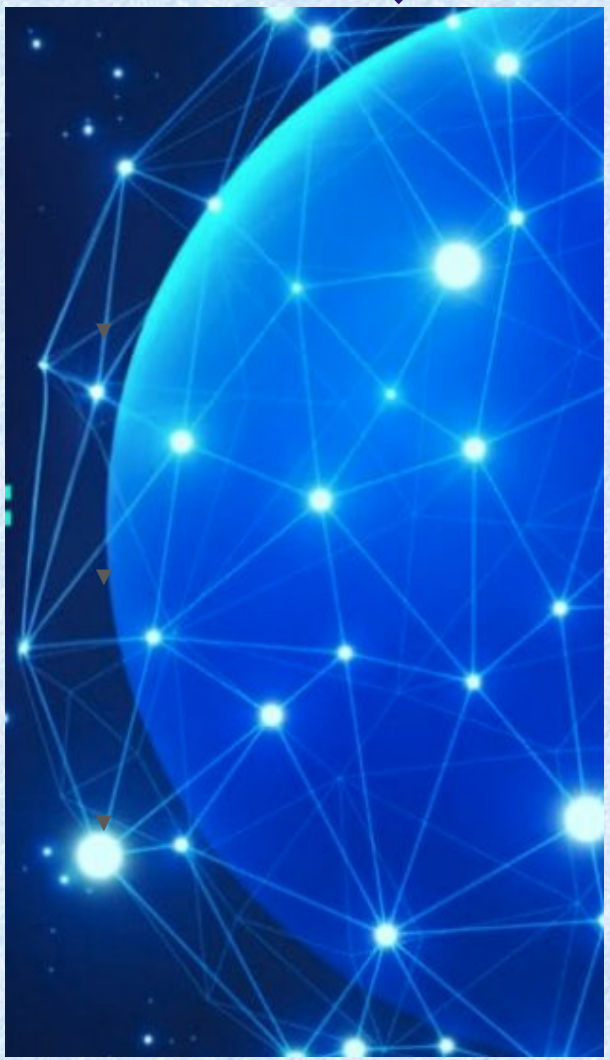
ENIGMA – GitHub Design – Python Mobility



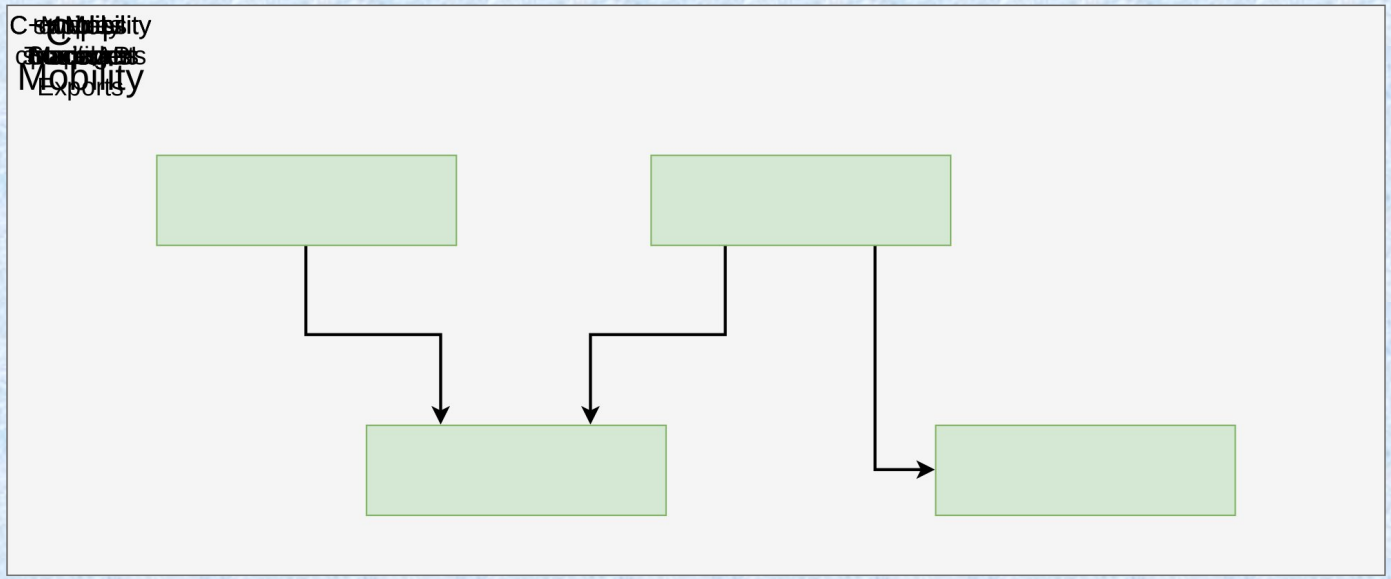


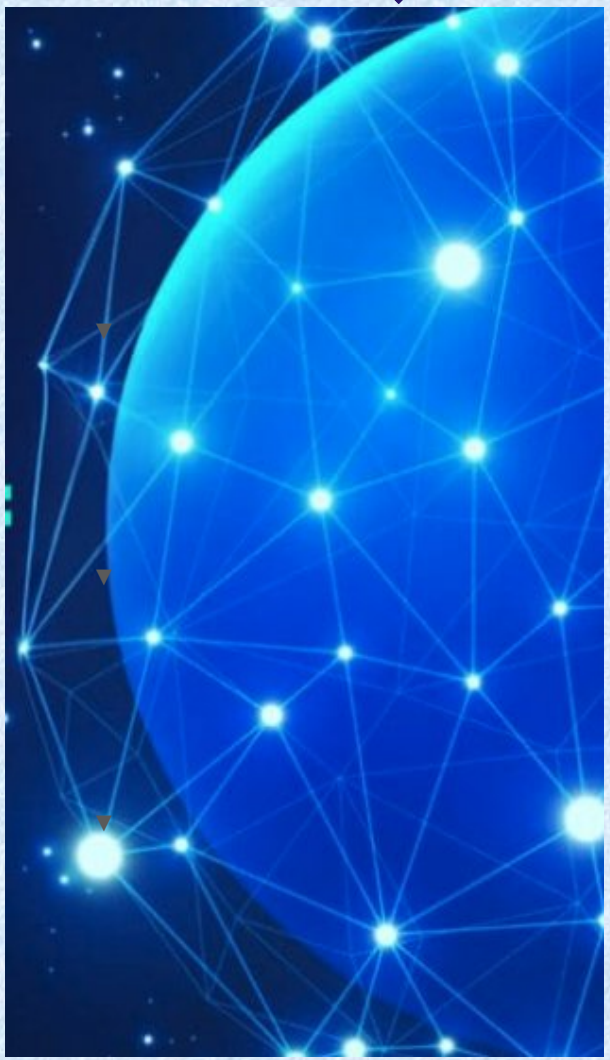
ENIGMA – GitHub Design – Simulation and Messaging



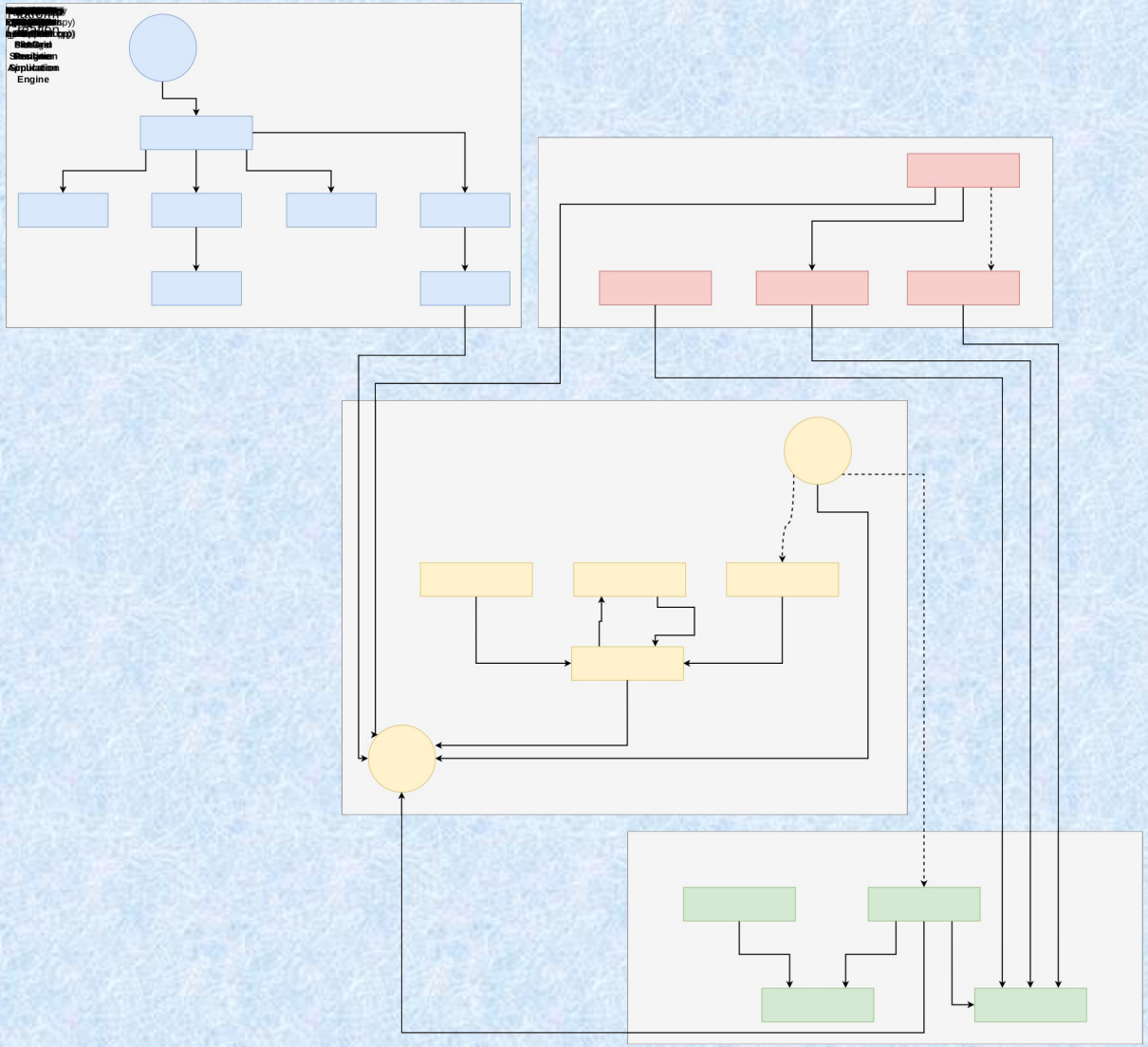


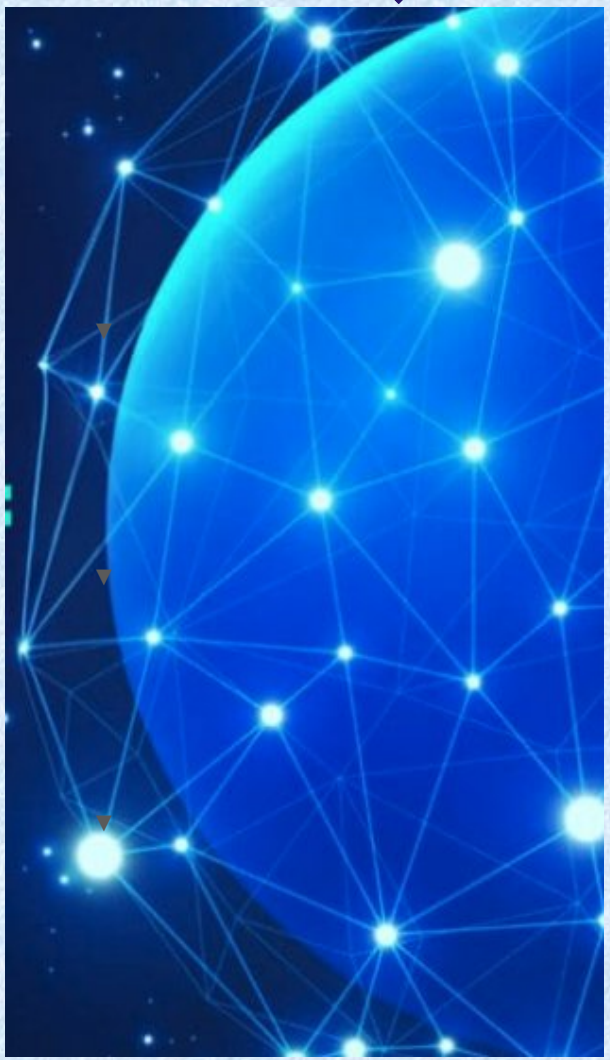
ENIGMA – GitHub Design – C++ Mobility Extension





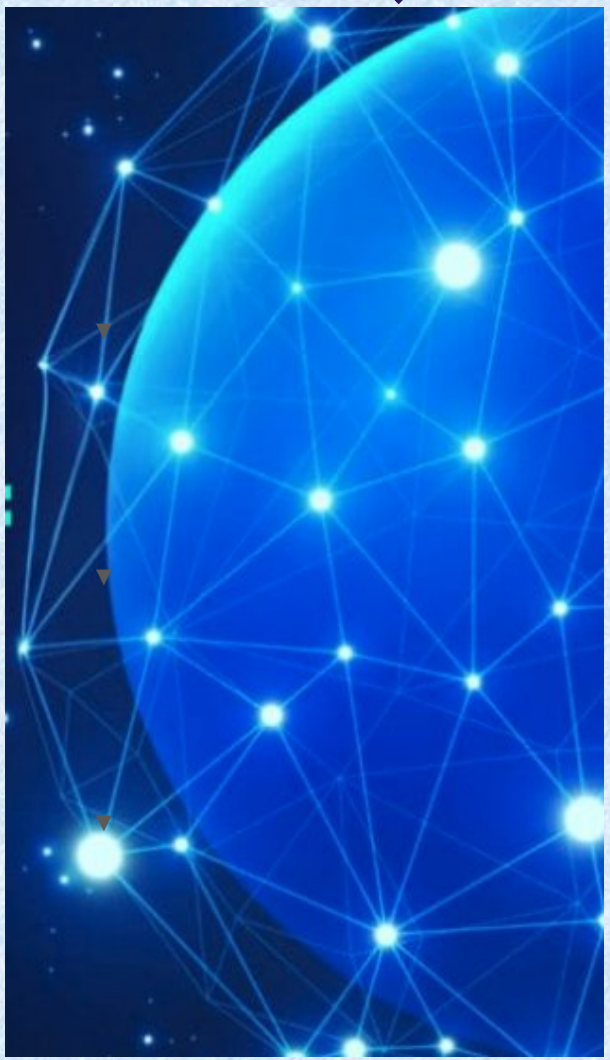
ENIGMA – GitHub Design





ENIGMA - SimGrid elements

- Platform — infrastructure: hosts (speed, cores), links (bandwidth, latency), zones + routing. XML.
- Actor — a concurrent process pinned to a host; your logic: execute() / put() / get() / sleep_for().
- Mailbox — named rendez-vous point for actor-to-actor messages; transfer cost derived from the platform.
- Engine — loads the platform, hosts the actors, runs the event loop, reports the simulated clock.



ENIGMA - How it works - C++ version

Build (CMake)

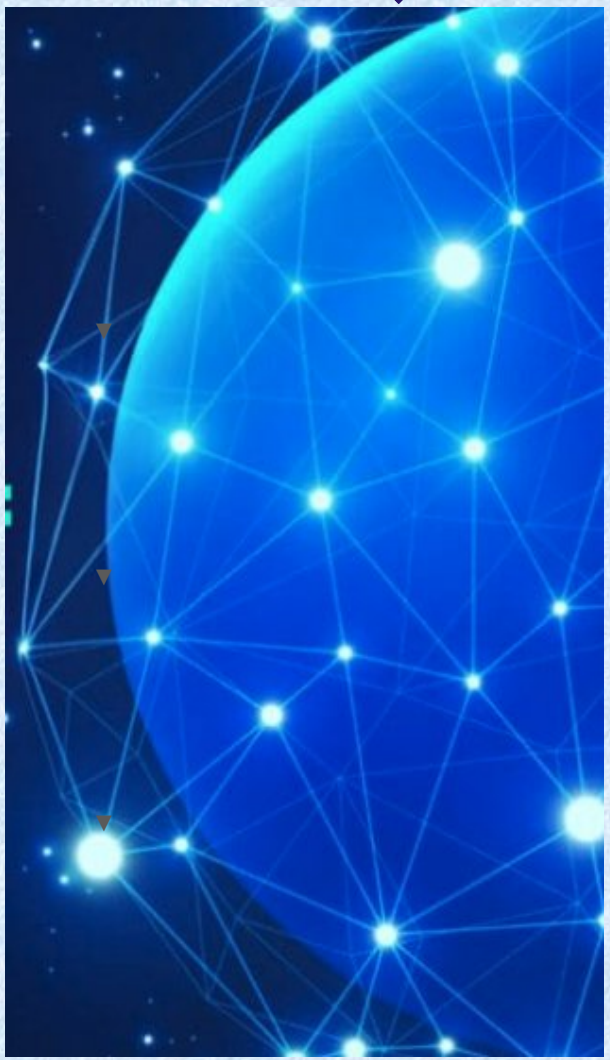
```
find_package(SimGrid 4.1)
add_library(enigma_platform STATIC ...)
add_library(enigma_mqtt     STATIC ...)
add_library(enigma_mobility STATIC ...)

add_executable(my_app tests/my_app.cpp)
target_link_libraries(my_app
   enigma_platform enigma_mqtt
   enigma_mobility ${SimGrid_LIBRARY})
```

Entry point

```
#include <simgrid/s4u.hpp>
namespace sg4 = simgrid::s4u;
XBT_LOG_NEW_DEFAULT_CATEGORY(ap
p,"app");

int main(int argc, char* argv[]) {
  sg4::Engine e(&argc, argv);
  e.load_platform(argv[1]);
  for (auto* h : e.get_all_hosts())
    h->add_actor("w", Worker());
  e.run();
}
```



ENIGMA - How it works - Python version

Setup

```
# SimGrid bindings ship with
# libsimgrid-dev
import sys, simgrid
sys.path.insert(0, 'src/python')

from enigma.mobility import (
    MobilityManager, MobilityVisualizer)
from enigma.comms.mqtt import (
    MQTTBroker, MQTTPublisher)
```

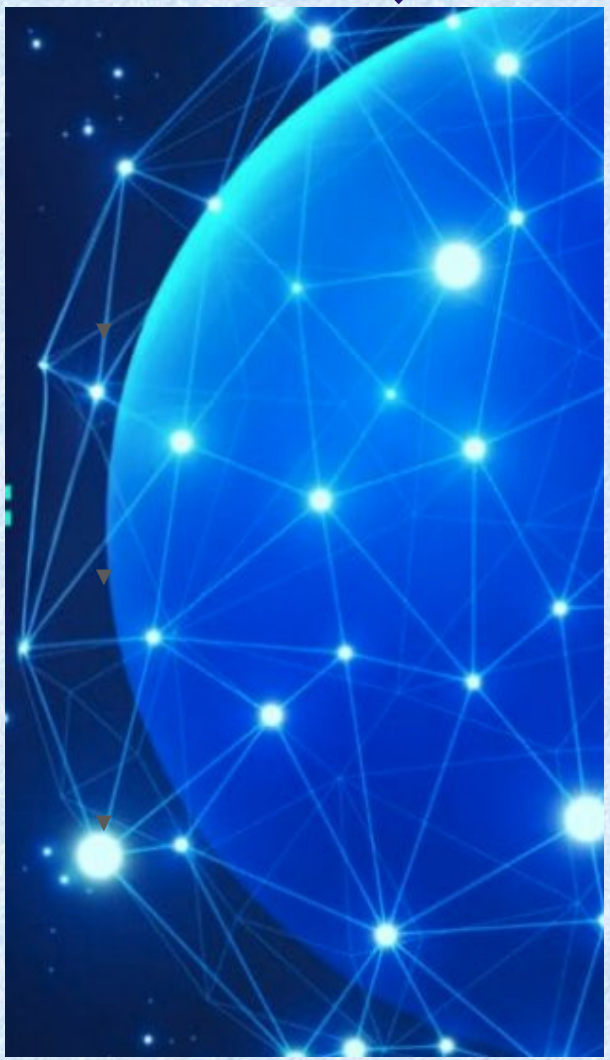
enigma/ mirrors the C++ modules.

Entry point

```
def worker():
    simgrid.this_actor.execute(1e9)

e = simgrid.Engine(sys.argv)
e.load_platform(sys.argv[1])
for h in e.all_hosts:
    simgrid.Actor.create('w', h, worker)
e.run()
```

Same XML, same coords/*.csv.



ENIGMA - C++ vs Python — the same actor

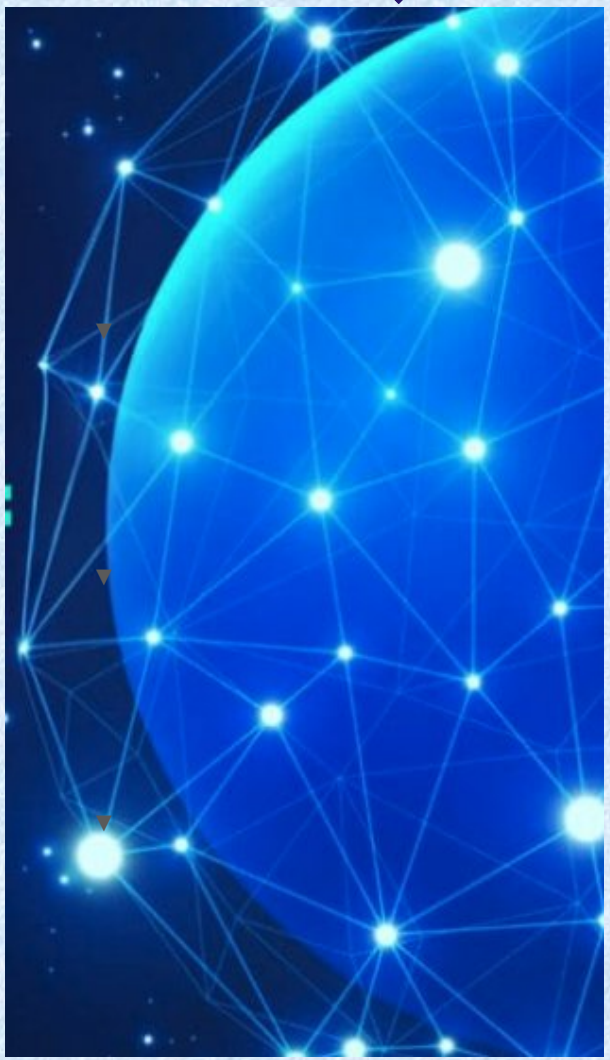
C++ (tests/*.cpp)

```
struct Worker {
  void operator()() {
    auto* h =
      sg4::this_actor::get_host();
    sg4::this_actor::sleep_for(1.0);
    sg4::this_actor::execute(1e9);
    sg4::Mailbox::by_name("gw")
      ->put(new std::string("hi"),
            1e6);
  }
};
```

Python (src/python/tests/*.py)

```
def worker():
    h = simgrid.this_actor.get_host()
    simgrid.this_actor.sleep_for(1.0)
    simgrid.this_actor.execute(1e9)
    simgrid.Mailbox.by_name('gw') \
        .put('hi', 1_000_000)
```

Mailbox names, flops and byte sizes are identical across both APIs.



ENIGMA - API - Generating a platform

C++

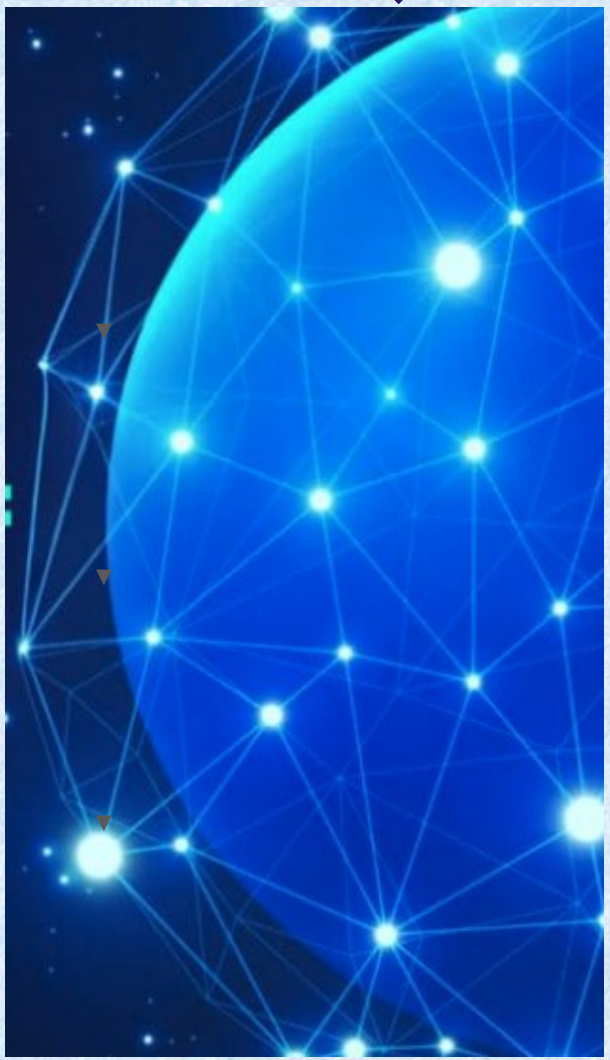
```
#include "platform/PlatformBuilder.hpp"
enigma::PlatformBuilder b;
b.createEdgeFogCloud("myplat")
 .addEdgeLayer (15, "2Gf",  "200MBps")
 .addFogLayer  ( 8, "15Gf", "2GBps")
 .addCloudLayer( 5, "150Gf","20GBps")
 .buildToFile("platforms/myplat.xml");
```

CLI tool

```
platform_generator edge  10
platform_generator iot   30 10
platform_generator edge-cluster 3 5

platform_generator hybrid-cluster \
  2 10  2 5  1 20  1 out.xml \
  --generate-app
```

--generate-app writes a tests/<name>_app.cpp template with TODO hooks.



ENIGMA - API – Using the SimGrid core

C++

```
sg4::Engine e(&argc, argv);
e.load_platform(xml);
e.get_all_hosts();
host->add_actor(name, Callable);

sg4::this_actor::execute(flops);
sg4::this_actor::sleep_for(s);
sg4::this_actor::get_host();

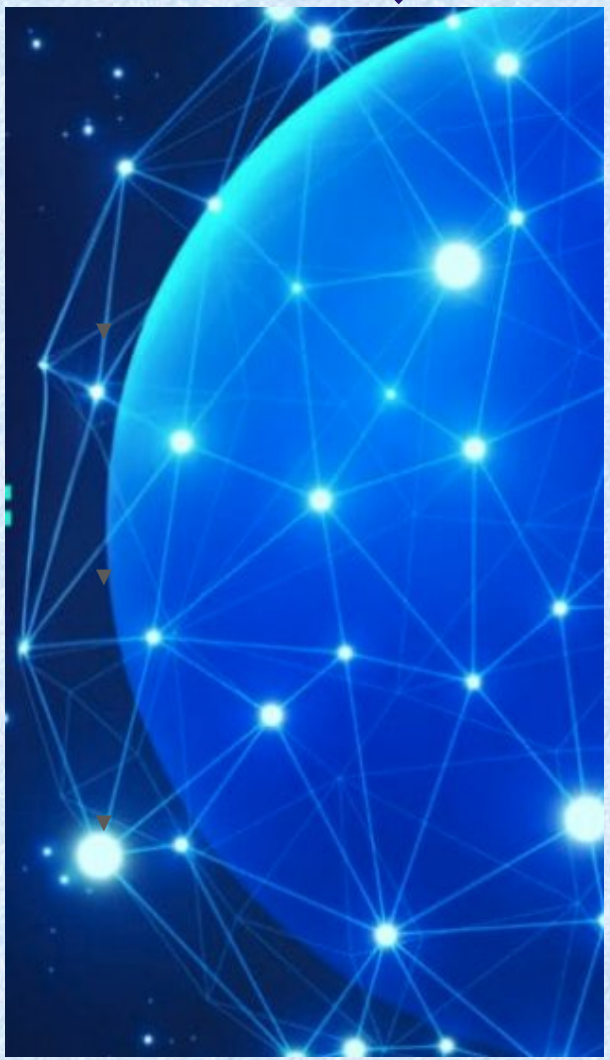
auto* m = sg4::Mailbox::by_name(n);
m->put(ptr, bytes);  m->get<T>();
```

Python

```
e = simgrid.Engine(sys.argv)
e.load_platform(xml)
e.all_hosts
simgrid.Actor.create(name, host, fn)

simgrid.this_actor.execute(flops)
simgrid.this_actor.sleep_for(s)
simgrid.this_actor.get_host()

m = simgrid.Mailbox.by_name(n)
m.put(obj, bytes);  m.get()
```



ENIGMA - API – The MQTT module (enigma_mqtt)

C++

```
#include "comms/mqtt/MQTT.hpp"
start_broker(host, "mqtt_broker");

MQTTPublisher pub("mqtt_broker","s1");
pub.publish("sensors/temp","25.5",100);

MQTTSubscriber sub("mqtt_broker");
sub.subscribe("sensors/#");
auto msg = sub.receive(10.0);
```

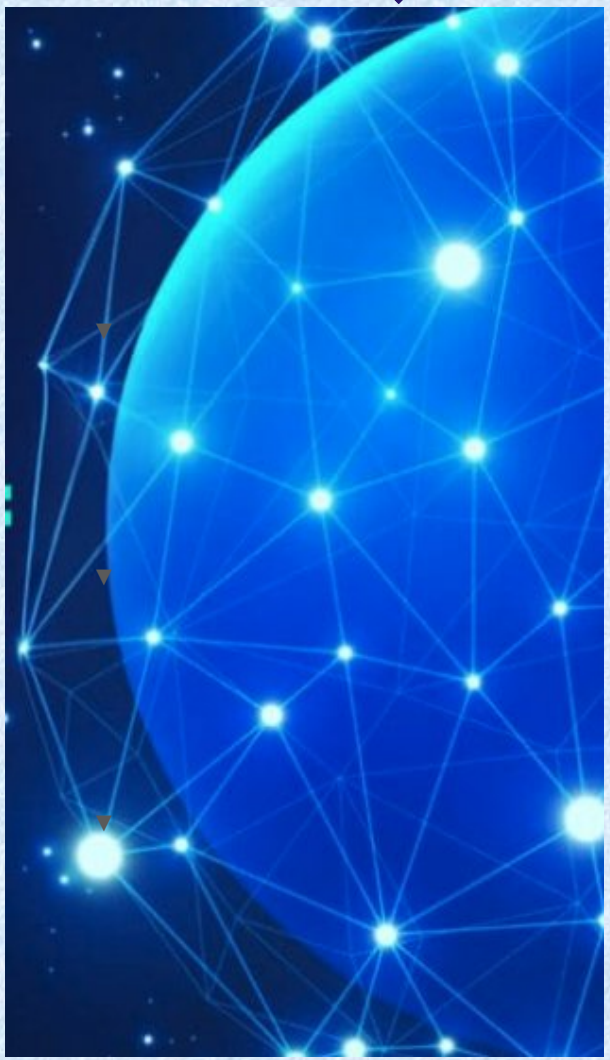
Python

```
from enigma.comms.mqtt import (
  MQTTBroker, MQTTPublisher,
  MQTTSubscriber)

pub = MQTTPublisher('mqtt_broker')
pub.publish('sensors/temp','25.5')

sub = MQTTSubscriber('mqtt_broker')
sub.subscribe('sensors/#')
```

QoS 0/1/2; topics support + and # wildcards.



ENIGMA - API – The Mobility module (enigma_mobility)

C++

```
#include "mobility/MobilityManager.hpp"
using namespace enigma::mobility;

MobilityManager mob(e);   // mobility_dir
mob.trace_count();
mob.has_trace("tgv_est");
mob.start_periodic_actor(e, 0.5);

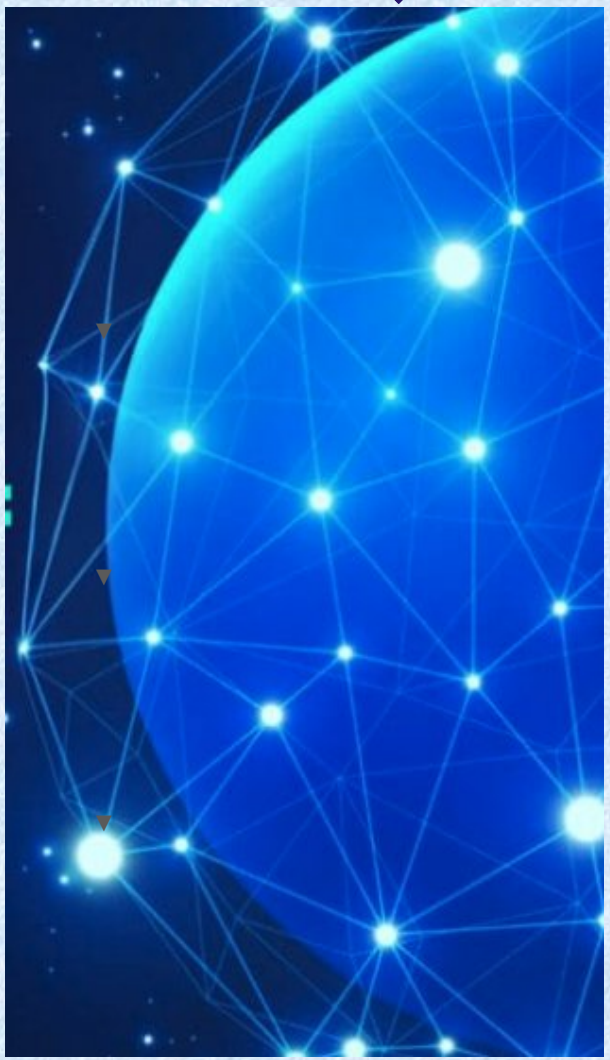
auto p = mob.position_at("tgv_est", t);
p->latitude;  p->extra["speed"];
mob.export_json("snap.json");
```

Python + Visualize

```
from enigma.mobility import (
  MobilityManager, MobilityVisualizer)

mob = MobilityManager(e,
        platform_xml='p.xml')
mob.start_periodic_actor(e,
        interval_s=0.5)
pos = mob.record('tgv_est',
        simgrid.Engine.clock)

MobilityVisualizer.save_interactive_map(
  mob.recorder,'map.html',offline=True)
```



ENIGMA - Requirements before using ENIGMA

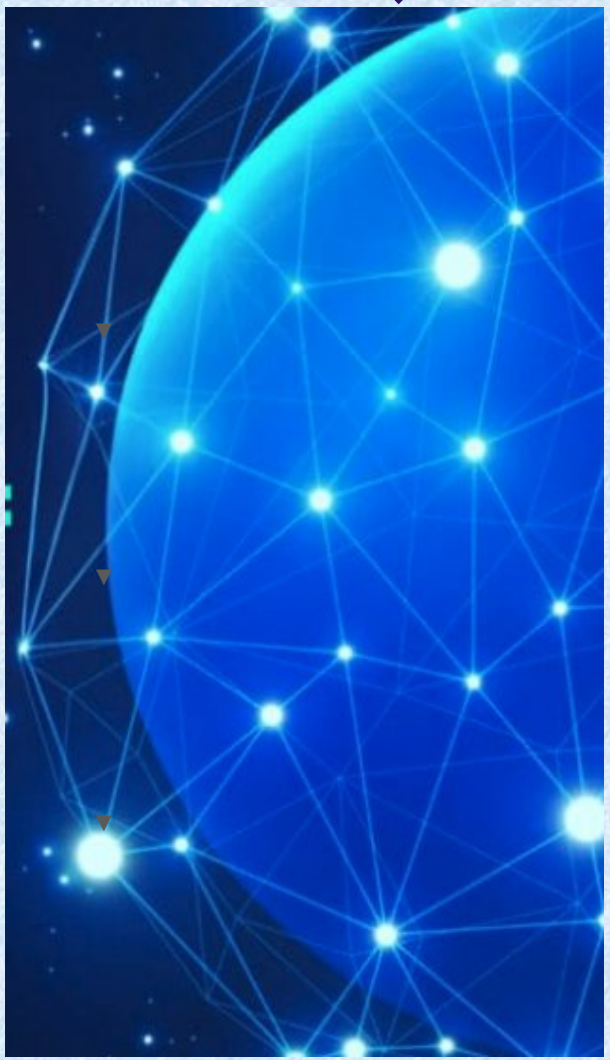
Mandatory

- C++17 compiler (GCC 7+, Clang 5+, MSVC 2017+) · CMake 3.10+ · SimGrid 4.1+ (libsimgrid-dev)

Optional — Python simulations & mobility visualization

- Python 3.8+ with the SimGrid Python bindings (shipped in libsimgrid-dev)
- folium — interactive map generation

playwright + chromium — live browser visualization



ENIGMA – Option 1 – Manual Installation

Ubuntu / Debian

- `sudo apt-get update`
- `sudo apt-get install -y simgrid libsimgrid-dev cmake build-essential`

From source

- `git clone https://github.com/simgrid/simgrid.git`
- `cd simgrid && mkdir build && cd build`
- `cmake -DCMAKE_INSTALL_PREFIX=/opt/simgrid-4.1 ..`
- `make -j$(nproc) && sudo make install`

ENIGMA auto-detects SimGrid in the package path or in /opt/simgrid-4.1.

DockerHub Repository

GitHub Repository

ENIGMA - Step 2 – Building ENIGMA

One command

```
git clone https://github.com/ENIGMA-Sim/ENIGMA.git
cd ENIGMA
./build.sh
```

or manually

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_PREFIX_PATH=/opt/simgrid-4.1
make -j$(nproc)
```

Produced in build/

- platform_generator
- edge_computing_app · fog_analytics_app
- hybrid_cloud_app · data_offloading_app
- mqtt_edge_app · mobility_test_app

Binaries need LD_LIBRARY_PATH=/opt/simgrid-4.1/lib at run time — the use-case scripts set it for you.

DockerHub Repository

GitHub Repository

ENIGMA – Option 2 – Using DockerHub

```
docker pull edelpozop/enigma:latest

# use cases 1-5: results in the terminal
docker run --rm edelpozop/enigma \
  ./use_cases/01_edge_computing/run.sh

# mobility: mount out/ to keep the files
docker run --rm -v "$PWD/out:/out" \
  -e OUT_PREFIX=/out/uc7 edelpozop/enigma \
  ./use_cases/07_mobility_france_trains/run.sh
```

```
# interactive map -> open it on the host

docker run --rm -v "$PWD/out:/out" \
  edelpozop/enigma python3 \
  src/python/tools/mobility_viewer.py \
  /out/uc7_snapshots.json --offline \
  --no-browser --save /out/uc7_map.html

# shell in /enigma: edit + recompile

docker run --rm -it edelpozop/enigma
cmake --build build -j"$(nproc)"
```

Files in out/ belong to root: sudo chown -R "$USER" out/. Native install → next slides.

DockerHub Repository

GitHub Repository

ENIGMA - Repository Layout

- include/ public headers (platform/ comms/mqtt/ mobility/ utils/)
- src/ implementations + src/tools/platform_generator
- src/python/ mirrored Python API (enigma.mobility, enigma.comms)
- tests/ example applications (*.cpp)
- platforms/ generated platform XML + coords*/ GPS traces
- use_cases/ <-- ready-to-run tutorial scenarios (01..07)

- build.sh run_examples.sh CMakeLists.txt



Stage 3. State-of-the-Art Use Cases. Hands-on Practice



PhD. Elias Del Pozo

Departamento de Arquitectura y Tecnología de Sistemas InformáticosUniversidad Politecnica de Madrid, Spain
e.delpozo@upm.es

Prof. Felix Garcia Carballeira

Computer Science and Engineering Department
University Carlos III of Madrid, Spain
fgcarbal@inf.uc3m.es

ENIGMA – The use_cases/folder

Each scenario is self-contained: generates its platform, runs a simulation, prints results.

use_cases/

- common.sh — SimGrid detection + incremental build helpers
- 01_edge_computing/ — local processing + gateway aggregation
- 02_fog_analytics/ — edge -> fog streaming analytics
- 03_hybrid_edge_fog_cloud/ — 3-tier Edge -> Fog -> Cloud pipeline
- 04_data_offloading/ — run-time local / fog / cloud decisions
- 05_mqtt_iot/ — MQTT publish / subscribe
- 06_mobility_madrid/ — Madrid buses / Metro / Cercanias + drones
- 07_mobility_france_trains/ — 5 TGV on real SNCF corridors (step by step)

```
./use_cases/01_edge_computing/run.sh      # build (once) + generate + run
```



ENIGMA – Use Case 1 – Edge Computing



- calls: platform_generator edge-cluster 3 5
- 3 edge clusters × 5 nodes = 15 hosts, 1 Gf / 125 MBps each
- platform_generator stamps every host / cluster with SimGrid host_energy properties automatically
- one host is picked as the gateway; the rest act as devices

use_cases/01_edge_computing/edge_computing.cpp

What happens, in order:

1. Device: collect sensor data (sleep 1 s)
2. Device: process locally (execute 1 GFlop)
3. Device: send the result to the gateway mailbox (1 MB)
4. Gateway: receive from every device, one by one
5. Gateway: aggregate each result (execute 500 MFlop)

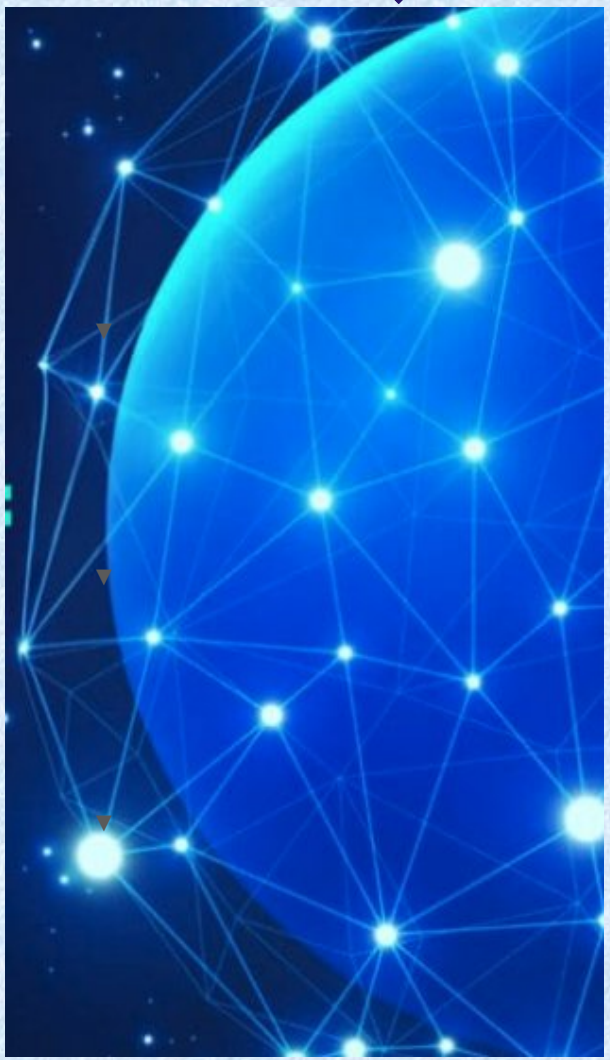
ENIGMA – Use Case 1 – Edge Computing

tests/edge_computing.cpp

```
class EdgeDevice {
  std::string gw_;
public:
  void operator()() {
    sg4::this_actor::sleep_for(1.0);
    sg4::this_actor::execute(1e9);
    sg4::Mailbox::by_name(gw_)
      ->put(new std::string("done"),
            1e6);
  }
};
// EdgeGateway: get<>() x N + execute(5e8)
```

Platform (Edge-cluster 3 5)

```
<zone id="edge_platform" routing="Full">
 <zone id="edge_cluster_0_zone"
       routing="Cluster">
  <cluster id="edge_cluster_0"
    prefix="edge_cluster_0_node_"
    radical="0-4" speed="1Gf" core="1"
    bw="125MBps" lat="50us"
    bb_bw="1GBps" bb_lat="10us"
    router_id="edge_cluster_0_router"/>
 </zone> ...
 <zoneRoute src="..0_zone" dst="..1_zone"/>
</zone>
```



ENIGMA – Use Case 2 – Fog Analytics



- calls: platform_generator fog-cluster 4 3
- 4 fog clusters × 3 nodes = 12 hosts, 10 Gf / 1 GBps, 4 cores
- ~1/3 of the hosts act as analyzers, the rest as data sources (round-robin)

use_cases/02_fog_analytics/fog_analytics.cpp

What happens, in order:

1. Source: sleep a random 0.5–2.0 s, then send a sample (10 KB)
2. Source: after N samples, send a FIN marker
3. Analyzer: receive each sample, run 100 MFlop analysis
4. Analyzer: once every source has sent FIN, run the aggregate pass
5. Analyzer: report count / mean / sum

ENIGMA – Use Case 2 – Fog Analytics

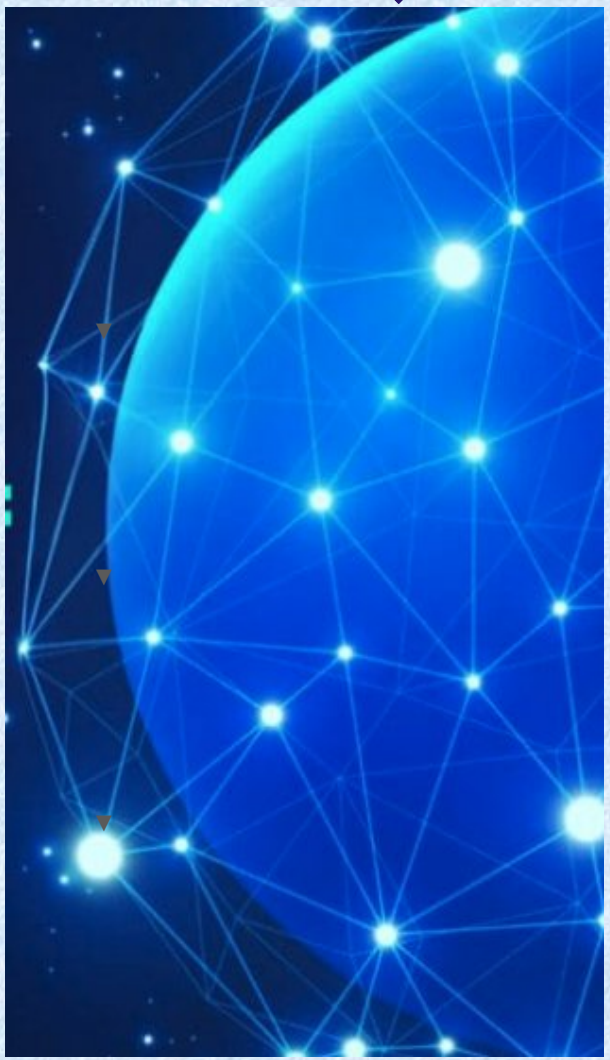
tests/fog_analytics.cpp

```
// DataSource: stream N samples, then FIN
for (int i=0;i<n;i++) {
  sg4::this_actor::sleep_for(w);
  fog_mbox->put(new double(v), 1e4);
}

// FogAnalyzer
while (done < sources) {
  auto* m = mbox->get<void>();
  sg4::this_actor::execute(1e8);
}
execute(all.size() * 5e7);  // aggregate
```

Platform (Fog-cluster 4 3)

```
<zone id="fog_platform"
      routing="Full">
 <zone id="fog_cluster_0_zone"
       routing="Cluster">
  <cluster id="fog_cluster_0"
    prefix="fog_cluster_0_node_"
    radical="0-2"
    speed="10Gf" core="4"
    bw="1GBps" lat="10us"/>
 </zone>
 ... x4 clusters + zoneRoutes
</zone>
```



ENIGMA – Use Case 3 – Hybrid Edge-Fog-Cloud



- calls: platform_generator hybrid-cluster 2 4 1 2 1 1 1
- 2 edge clusters × 4 nodes (1 Gf), 1 fog cluster × 2 nodes (10 Gf), 1 cloud cluster × 1 node (100 Gf)
- trailing 1 = direct Edge ↔ Cloud links (2 GBps / 30 ms) on top of Edge → Fog → Cloud
- hosts are classified by the substrings edge / fog / cloud in their names

use_cases/03_hybrid_edge_fog_cloud/hybrid_cloud.cpp

What happens, in order:

1. Edge: filter locally 3× (200 MFlop each), push 500 KB to fog
2. Fog: pre-process each edge message (500 MFlop)
3. Fog: aggregate (2 GFlop), push a 1 MB summary to cloud
4. Cloud: store each summary (1 GFlop)
5. Cloud: run the final analytics / ML pass (10 GFlop)

ENIGMA – Use Case 3 – Hybrid Edge-Fog-Cloud

tests/hybrid_cloud.cpp

```
// EdgeCollector
execute(2e8);  fog_mbox->put(d, 5e5);

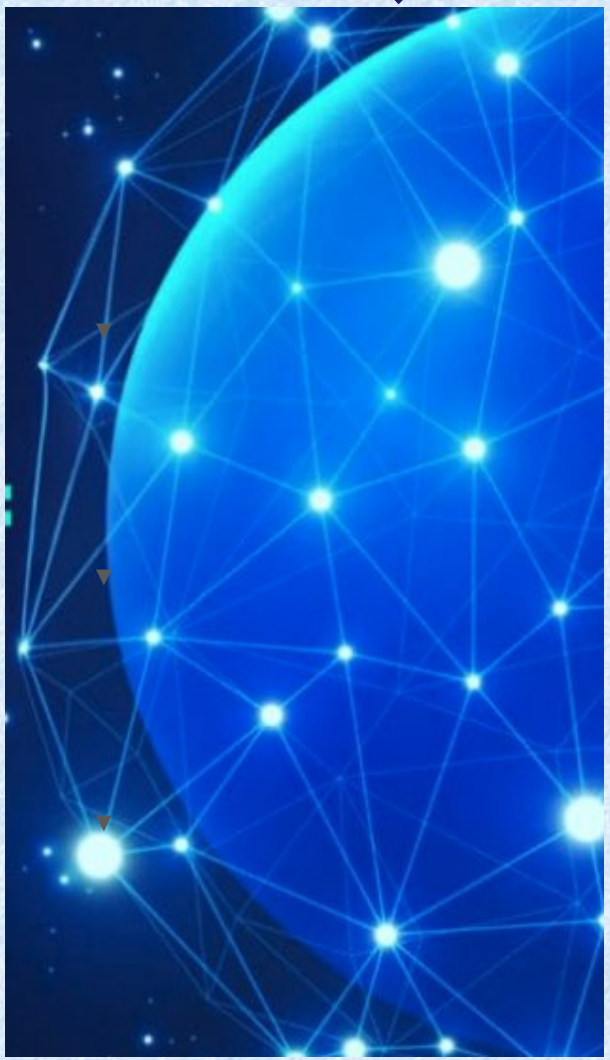
// FogAggregator
loop: mbox->get<std::string>();
execute(5e8); ... execute(2e9);
cloud_mbox->put(summary, 1e6);

// CloudProcessor
loop: mbox->get<std::string>();
execute(1e9); ... execute(10e9); // ML
```

Platform (hybrid-cluster 2 4 1 2 1 1 1)

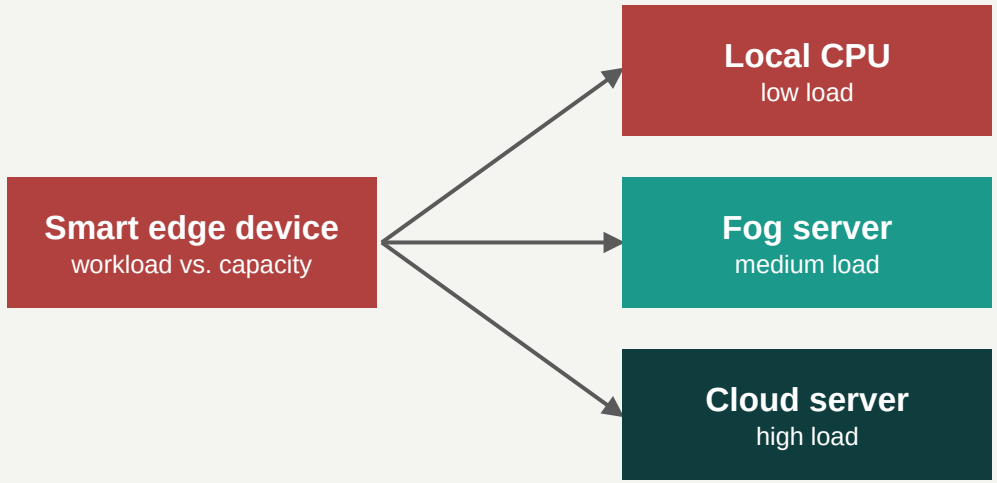
```
<zone id="hybrid_platform"
      routing="Full">
  edge_cluster_0..1_zone  (1Gf)
  fog_cluster_0_zone      (10Gf)
  cloud_cluster_0_zone    (100Gf)

  <link id="link_edge_..to_fog_.."
     bandwidth="500MBps" latency="10ms"/>
  <link id="link_fog_..to_cloud_.."
     bandwidth="5GBps"  latency="50ms"/>
  + direct edge<->cloud links (arg 7 = 1)
</zone>
```



ENIGMA – Use Case 4 – Smart Data Offloading

- calls: platform_generator hybrid-cluster 2 4 2 2 1 1 1
- 8 edge hosts, 4 fog hosts, 1 cloud host (same generator as UC3)
- devices are created with an increasing workload: 1e9 × (0.5 + i·0.5)



1. Device: compare its own CPU capacity against the task workload
   a) Low load → execute locally, no network involved
   b) Medium load → send {workload, reply_mbox} to a fog server
   c) High load → send {workload, reply_mbox} to a cloud server
2. Server: execute the workload, reply with the measured latency; self-terminate after a 10 s idle timeout

ENIGMA – Use Case 4 – Smart Data Offloading

tests/data_offloading.cpp

```
double cap = host->get_speed();

if (workload < cap * 0.5)
      sg4::this_actor::execute(workload);   // local
else if (workload < cap * 2.0)
      fog_mbox->put({workload, reply});     // -> fog
else
      cloud_mbox->put({workload, reply});   // -> cloud

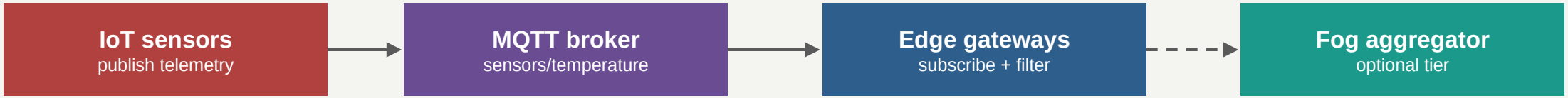
// server: get<pair>(10.0);
//   execute(workload); reply->put(latency)
```

run

```
gen_platform hybrid-cluster 2 4  2 2  1 1  1 hybrid_platform.xml

data_offloading_app  platforms/hybrid_platform.xml  5
#                                                    ^
#                                                    tasks per edge device
```

ENIGMA – Use Case 5 – MQTT for IoT



- calls: platform_generator edge-cluster 2 4 → 8 hosts named edge_cluster_*
- the app auto-detects the roles: host 0 → broker, next 2 → sensors, rest → gateways
- no fog tier here, so gateways process locally

1. Broker: starts on host 0, loops forever accepting connections
2. Sensors: publish 5 telemetry messages each to sensors/temperature
3. Gateways: subscribe to sensors/temperature, filter/process each message
4. If a fog node exists, gateways republish to edge/filtered

ENIGMA – Use Case 5 – MQTT for IoT

tests/mqtt_edge_app.cpp

```
#include "comms/mqtt/MQTT.hpp"

start_broker(broker_host, "mqtt_broker");

// IoTSensor
MQTTPublisher pub("mqtt_broker");
pub.publish("sensors/temperature",
            payload, 100);

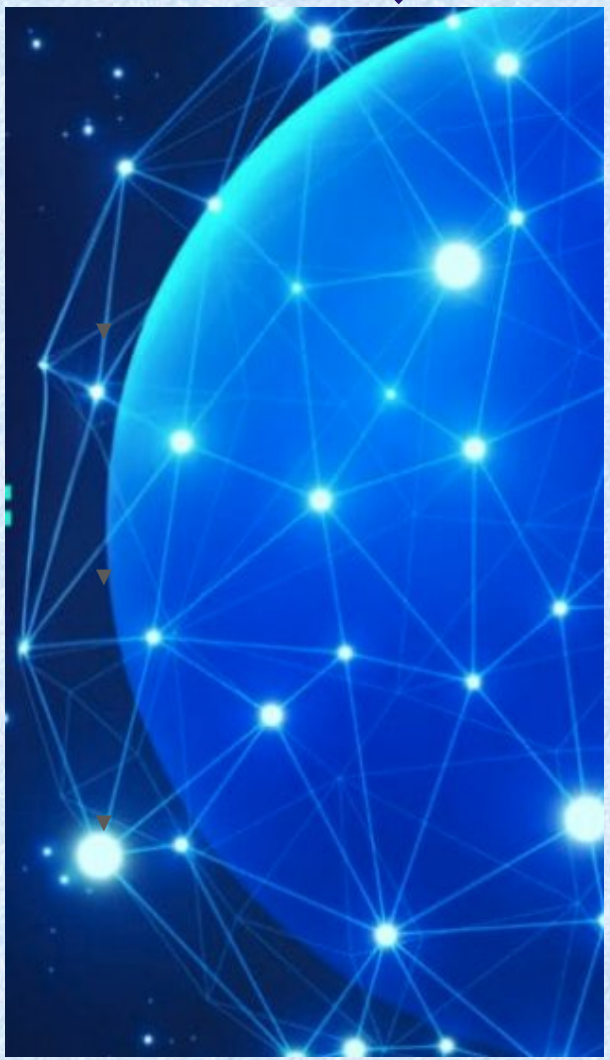
// EdgeGateway
MQTTSubscriber sub("mqtt_broker");
sub.subscribe("sensors/temperature");
auto msg = sub.receive();
```

platform (Edge-cluster 2 4)

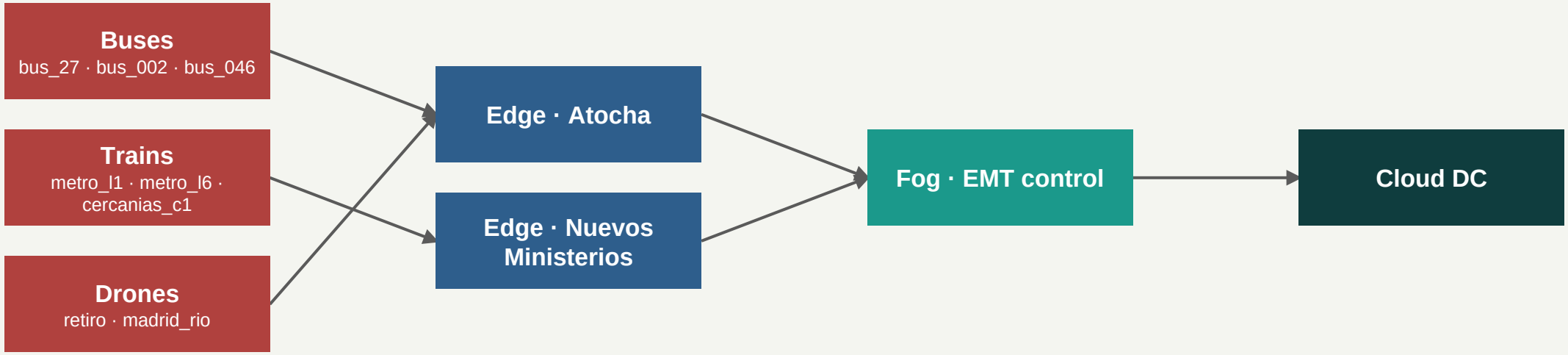
```
# 8 hosts named edge_cluster_*
#   host 0  -> MQTT broker
#   next 2  -> sensors
#   rest    -> gateways

<cluster id="edge_cluster_0"
  prefix="edge_cluster_0_node_"
  radical="0-3" speed="1Gf"
  bw="125MBps" lat="50us"/>
```

Broker loops forever — the '1 actor still active' line is expected.



ENIGMA – Use Case 6 – Madrid Public-Transport Mobility



1. Activate the energy plugin, then load the platform
2. MobilityManager loads every CSV trace in coords/
3. A periodic actor snapshots every device's position every 0.5 sim-s
4. Each mobile actor queries its interpolated position and does a bit of local work
5. Export snapshots to JSON/CSV once e.run() returns

ENIGMA – Use Case 6 – Madrid Public-Transport Mobility

Concept

- Buses: bus_27 · bus_002 · bus_046 (EMT corridors)
- Trains: metro_l1 · metro_l6 (Circular) · cercanias_c1
- Drones: drone_retiro · drone_madrid_rio
- generate_traces.py holds real Madrid waypoint tables

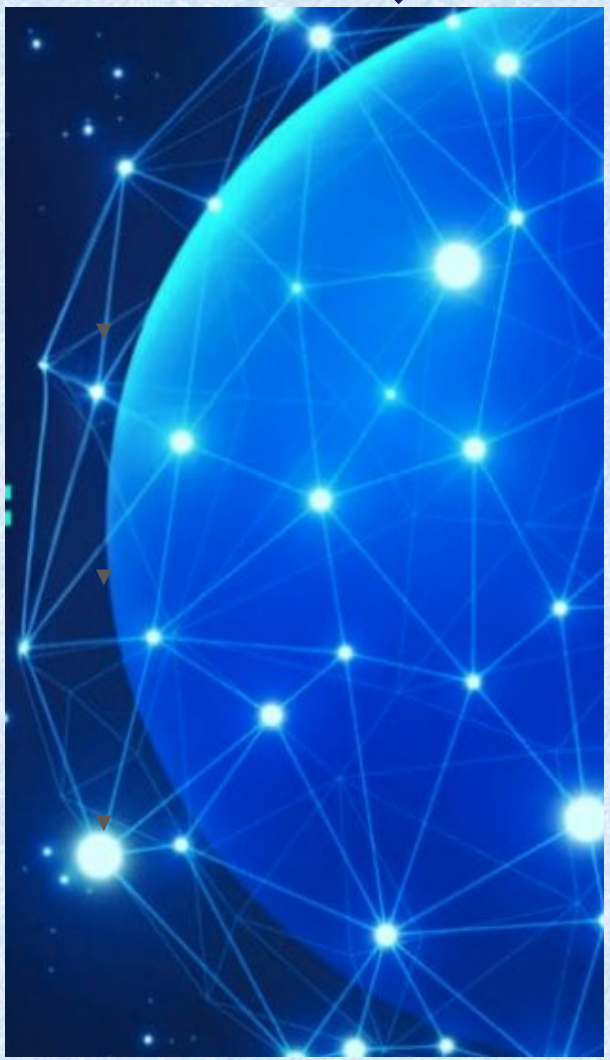
coords/metro_l6.csv

```
timestamp,latitude,longitude,
  speed,heading,occupancy
0,40.4468,-3.7038,14.0,0.0,55.0
```

madrid_transport_platform.xml

```
<zone id="madrid_transport"
      routing="Full">
  <prop id="mobility_dir"
    value="use_cases/06_mobility_madrid/
          coords/"/>

  <host id="bus_27"   speed="1Gf"/>
  <host id="metro_l6" speed="1500Mf"/>
  ...
  <route src="metro_l6"
         dst="edge_nuevos_ministerios">
    <link_ctn id="lte_metro_l6"/>
  </route>
</zone>
```



ENIGMA – Use Case 6 – Steps for running using DockerHub

- docker pull enigmasim/enigma:tagname
- In the directory where you downloaded the project: mkdir -p out
- docker run --rm -v "$PWD/out:/out" enigmasim/enigma bash -c "OUT_PREFIX=/out/uc6 ./use_cases/06_mobility_madrid/run.sh"

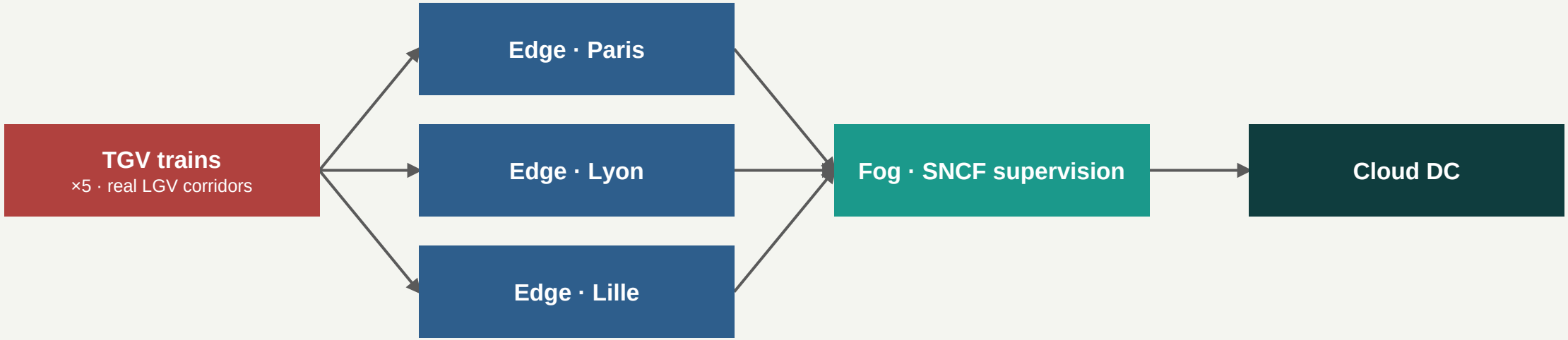
- docker run --rm -v "$PWD/out:/out" enigmasim/enigma python3 src/python/tools/mobility_viewer.py /out/uc6_snapshots.json --offline --no-browser --save /out/uc6_map.html

- open out/uc6_map.html # MacOS
- xdg-open out/uc6_map.html # Linux
- start out\uc6_map.html # Windows

ENIGMA – Use Case 6



ENIGMA – Use Case 7 – France TGV Mobility



1. Activate the energy plugin, then load the platform
2. MobilityManager loads every CSV trace in coords/
3. A periodic actor snapshots every device's position every 0.5 sim-s
4. Each mobile actor queries its interpolated position and does a bit of local work
5. Export snapshots to JSON/CSV once e.run() returns

ENIGMA – Use Case 7 – France TGV Mobility

5 trains along real SNCF LGV corridors

- tgv_sud_est — Paris Gare de Lyon → Lyon → Marseille (~686 km)
- tgv_atlantique — Paris Montparnasse → Tours → Bordeaux (~507 km)
- tgv_nord — Paris Gare du Nord → Lille → Calais (~297 km)
- tgv_est — Paris Gare de l'Est → Reims → Strasbourg (~409 km)
- tgv_rhin_rhone — Dijon → Besançon → Belfort → Mulhouse (~185 km)



ENIGMA – Use Case 7 – Stage 1

france_rail_platform.xml

- SimGrid 4.1, routing="Full"
- one <host> per train + fixed infra
- links: backbone / fog uplink / per-train GSM-R

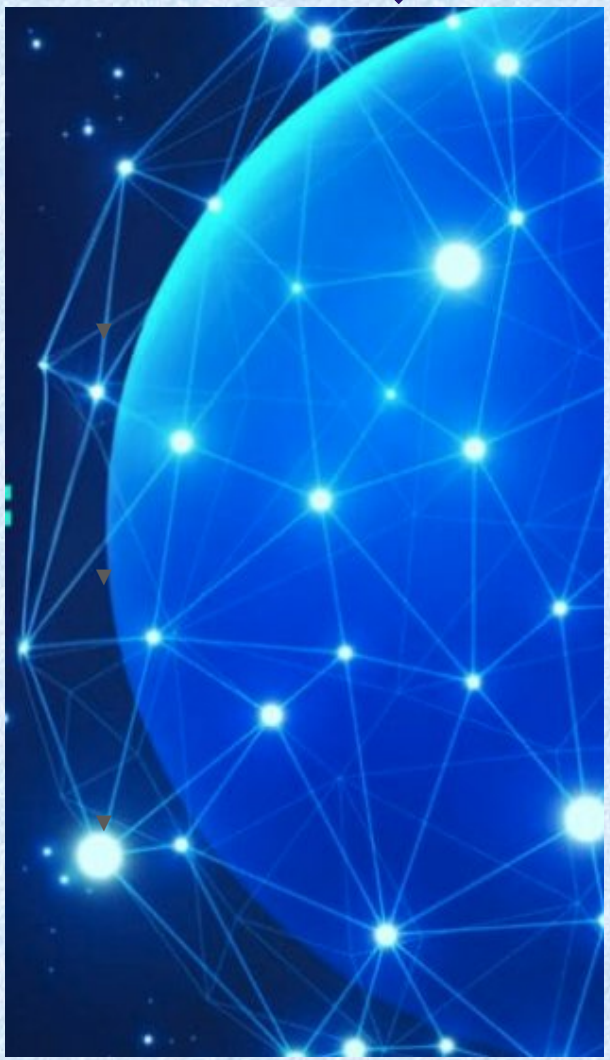
mobility switched on by ONE property

platform.xml

```
<zone id="france_rail" routing="Full">
  <prop id="mobility_dir"
    value="use_cases/
      07_mobility_france_trains/coords/"/>

  <host id="edge_paris" speed="4Gf"/>
  <host id="fog_sncf"   speed="25Gf"/>
  <host id="tgv_sud_est" speed="2Gf"/>
  ... x5 trains

  <route src="tgv_sud_est" dst="edge_lyon">
    <link_ctn id="gsmr_sud_est"/>
    </route>
</zone>
```



ENIGMA – Use Case 7 – Stage 2

generate_traces.py (stdlib only)

```
TGV_SUD_EST = [
  (48.84437, 2.37344),  # Paris Gare de Lyon
  (45.76054, 4.85953),  # Lyon Part-Dieu
  (43.30370, 5.38030),  # Marseille St-Charles
]
ROUTES = { 'tgv_sud_est':
   dict(wp=TGV_SUD_EST, cruise=92.0) }

# walk the polyline at cruise m/s,
# ping-pong at the ends, sample every 1 s
```

coords/tgv_sud_est.csv

```
timestamp,latitude,longitude,speed,heading,pax_load,delay_s
0,48.844370,2.373440,92.0,0.0,60.0,30
1,48.843546,2.373701,92.0,197.5,60.1,30
...
```

ENIGMA – Use Case 7 – Stage 3

tests/mobility_test.cpp

```
MobilityManager mob(e);  // mobility_dir
mob.start_periodic_actor(e, 0.5);

for (auto* h : e.get_all_hosts())
  if (mob.has_trace(h->get_cname()))
    h->add_actor("mobile_actor",
      MobileActor(h->get_cname(), &mob));

e.run();
mob.export_json("snap.json");
mob.export_csv ("snap.csv");
```

inside MobileActor

```
double t = sg4::Engine::get_clock();
auto pos = mob->position_at(name, t);

if (pos) {
            double load =    pos->extra["pax_load"];
            double dly =      pos->extra["delay_s"];
}
sg4::this_actor::execute(5e8);
```

The periodic actor records every train every 0.5 sim-s → snapshots.

ENIGMA – Use Case 7 – Stage 4

Run

```
./use_cases/
  07_mobility_france_trains/run.sh

# -> /tmp/enigma_uc7_snapshots.json
#    /tmp/enigma_uc7_snapshots.csv
#    /tmp/enigma_uc7_raw_traces.json
#    (~72 000 snapshots, t = 7201 s)
```

Launch interactive Map

```
python3 src/python/tools/   mobility_viewer.py \
/tmp/enigma_uc7_snapshots.json   --offline
```

- one polyline per corridor on OpenStreetMap
- time slider animates the 5 TGV

click a dot: speed / heading / pax_load / delay_s

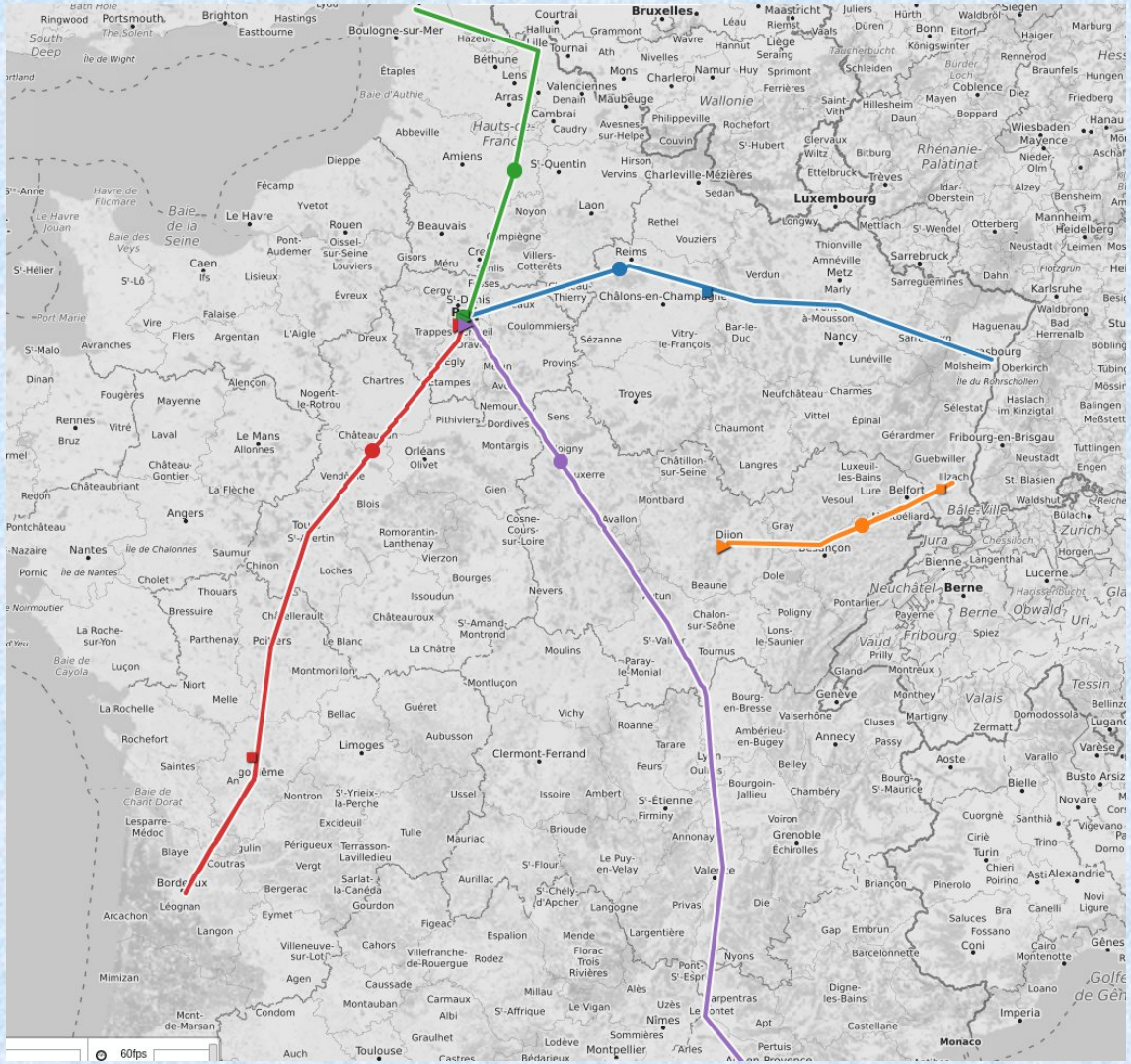
ENIGMA – Use Case 7 – Steps for running using DockerHub

- docker pull enigmasim/enigma:tagname
- In the directory where you downloaded the project: mkdir -p out
- docker run --rm -v "$PWD/out:/out" enigmasim/enigma bash -c "OUT_PREFIX=/out/uc7 ./use_cases/07_mobility_france_trains/run.sh"

- docker run --rm -v "$PWD/out:/out" enigmasim/enigma python3 src/python/tools/mobility_viewer.py /out/uc7_snapshots.json --offline --no-browser --save /out/uc7_map.html

- open out/uc7_map.html # MacOS
- xdg-open out/uc7_map.html # Linux
- start out\uc7_map.html # Windows

ENIGMA – Use Case 7



Summary

- ENIGMA = platform generation + example apps + MQTT + mobility, on top of SimGrid 4.1 — C++ and Python.
- use_cases/ 01–07 are one-command tutorials; 06 (Madrid) and 07 (France TGV) add GPS mobility with an interactive map.

Docs in the repo

- README.md · QUICKSTART.md · CLUSTER_USAGE.md · include/mobility/README.md · src/python/enigma/mobility/README.md

Publications

- FGCS 2023 — A Scalable Simulator for Cloud, Fog and Edge Computing with Mobility Support
- PDP 2023 — ENIGMA: A Scalable Simulator for IoT and Edge Computing

